

# BEE 4750 Lab 2: Monte Carlo

**Name:** William Natcharian

**ID:** wrn25

## **Due Date**

Thursday, 10/02/25, 9:00pm

## **Setup**

The following code should go at the top of most Julia scripts; it will load the local package environment and install any needed packages. You will see this often and shouldn't need to touch it.

```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
```

```
Activating project at `~/Desktop/4750 Cloned Repositories/lab-2-Wyndter`
  Installed Libmount_jll          v2.41.2+0
  Installed Libtiff_jll           v4.7.2+0
  Installed Libuuid_jll           v2.41.2+0
  Installed Xorg_xcb_util_cursor_jll v0.1.6+0
  Installed Xorg_libXfixes_jll     v6.0.2+0
  Installed FillArrays             v1.14.0
  Installed Plots                  v1.41.1
Precompiling project...
 409.0 ms  Libuuid_jll
 440.5 ms  Libmount_jll
 375.0 ms  Xorg_libXfixes_jll
 490.0 ms  Libtiff_jll
 742.8 ms  FillArrays
```

```

374.7 ms    Xorg_xcb_util_cursor_jll
710.4 ms    ColorVectorSpace → SpecialFunctionsExt
776.5 ms    Fontconfig_jll
563.6 ms    FillArrays → FillArraysStatisticsExt
578.2 ms    FillArrays → FillArraysSparseArraysExt
704.0 ms    FillArrays → FillArraysPDMatsExt
658.3 ms    Cairo_jll
877.0 ms    Qt6Base_jll
641.2 ms    HarfBuzz_jll
2777.8 ms   ColorSchemes
787.8 ms    Qt6ShaderTools_jll
624.0 ms    libass_jll
645.6 ms    Pango_jll
782.1 ms    FFMPEG_jll
1317.0 ms   Qt6Declarative_jll
654.0 ms    FFMPEG
1059.2 ms   GR_jll
3684.8 ms   Distributions
994.5 ms    Distributions → DistributionsTestExt
2320.6 ms   GR
5258.6 ms   PlotUtils
1726.3 ms   PlotThemes
2294.9 ms   RecipesPipeline
32716.5 ms  Plots
29 dependencies successfully precompiled in 44 seconds. 162 already precompiled.

```

```

using Random # random number generation
using Distributions # probability distributions and interface
using Statistics # basic statistical functions, including mean
using Plots # plotting

```

## Overview

In this lab, we will conduct a Monte Carlo experiment and explore how sensitive Monte Carlo estimates are to the underlying probability distribution(s).

You should always start any computing with random numbers by setting a “seed,” which controls the sequence of numbers which are generated (since these are not *really* random, just “pseudorandom”). In Julia, we do this with the `Random.seed!()` function.

```
Random.seed!(1)
```

TaskLocalRNG()

It doesn't matter what seed you set, though different seeds might result in slightly different values. But setting a seed means every time your notebook is run, the answer will be the same.

### Seeds and Reproducing Solutions

If you don't re-run your code in the same order or if you re-run the same cell repeatedly, you will not get the same solution. If you're working on a specific problem, you might want to re-use `Random.seed()` near any block of code you want to re-evaluate repeatedly.

### Probability Distributions and Julia

Julia provides a common interface for probability distributions with the [Distributions.jl package](#). The basic workflow for sampling from a distribution is:

1. Set up the distribution. The specific syntax depends on the distribution and what parameters are required, but the general call is the similar. For a normal distribution or a uniform distribution, the syntax is

```
# you don't have to name this "normal_distribution"
#   is the mean and   is the standard deviation
normal_distribution = Normal( , )
# a is the upper bound and b is the lower bound; these can be set to +Inf or -Inf for ar
uniform_distribution = Uniform(a, b)
```

There are lots of both [univariate](#) and [multivariate](#) distributions, as well as the ability to create your own, but we won't do anything too exotic here.

2. Draw samples. This uses the `rand()` command (which, when used without a distribution, just samples uniformly from the interval  $[0, 1]$ .) For example, to sample from our normal distribution above:

```
# draw n samples
rand(normal_distribution, n)
```

Putting this together, let's say that we wanted to simulate 100 six-sided dice rolls. We could use a [Discrete Uniform distribution](#).

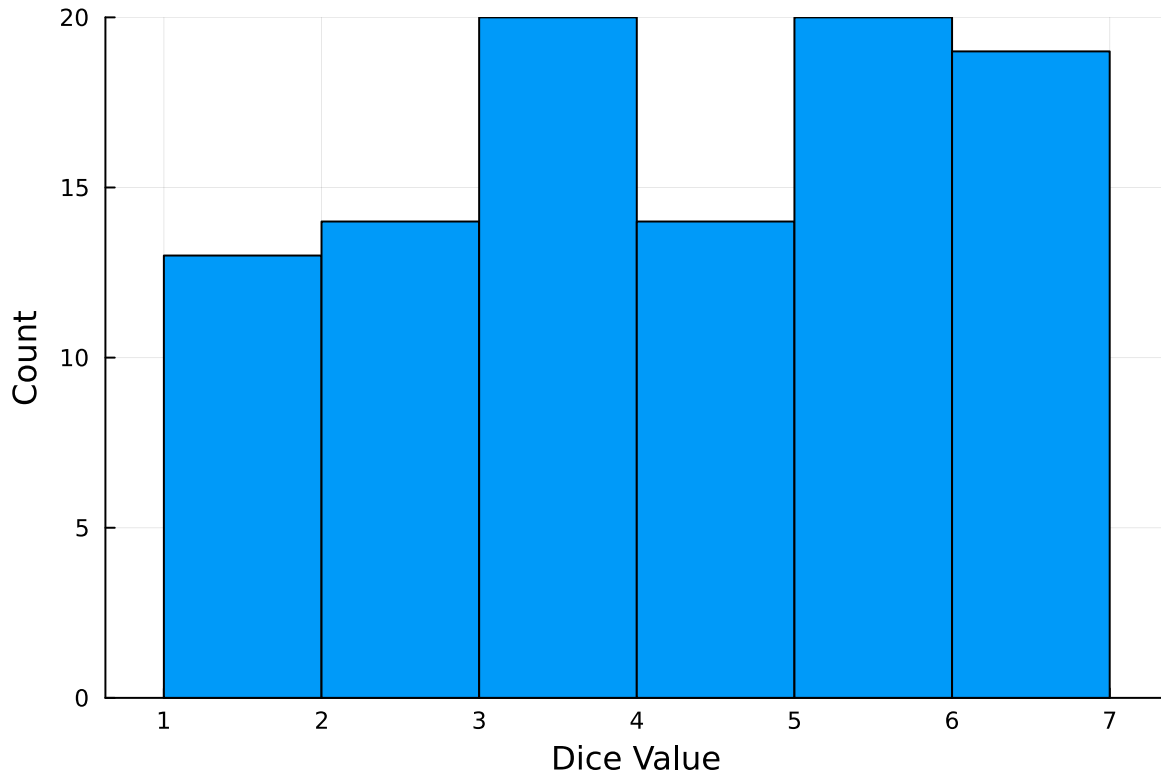
```
dice_dist = DiscreteUniform(1, 6) # can generate any integer between 1 and 6
dice_rolls = rand(dice_dist, 100) # simulate rolls
```

100-element Vector{Int64}:

1  
3  
5  
4  
6  
2  
5  
5  
5  
2  
  
3  
6  
5  
5  
6  
3  
6  
6  
6

And then we can plot a histogram of these rolls:

```
histogram(dice_rolls, legend=:false, bins=6)  
ylabel!("Count")  
xlabel!("Dice Value")
```



### Instructions

#### Remember to:

- Evaluate all of your code cells, in order (using a `Run All` command). This will make sure all output is visible and that the code cells were evaluated in the correct order.
- Tag each of the problems when you submit to Gradescope; a 10% penalty will be deducted if this is not done.

### Problem

A common engineering problem is to quantify flood risk, which is typically computed by propagating a flood hazard distribution through a *depth-damage function* relating flood depths to economic damages. A reasonable depth-damage function for a house without a basement is a bounded logistic function,

$$d(h) = \mathbb{1}_{h>0} \frac{L}{1 + \exp(-k(h - h_0))},$$

where  $d$  is the damage as a percent of total structure value,  $h$  is the water depth (relative to the house's ground floor elevation) in m,  $\mathbb{1}_{x>0}$  is the indicator function,  $L$  is the maximum loss in USD,  $k$  is the slope of the depth-damage relationship, and  $h_0$  is the inflection point. We'll assume  $L = \$200,000$ ,  $k = 0.8$ , and  $h_0 = 3$ .

For this problem, suppose that we have two different probability distributions characterizing annual maxima flood depths:

1.  $h \sim \text{LogNormal}(1.2, 0.3)$ ;
2.  $h \sim \text{GeneralizedExtremeValue}(3, 0.9, -0.15)$ .

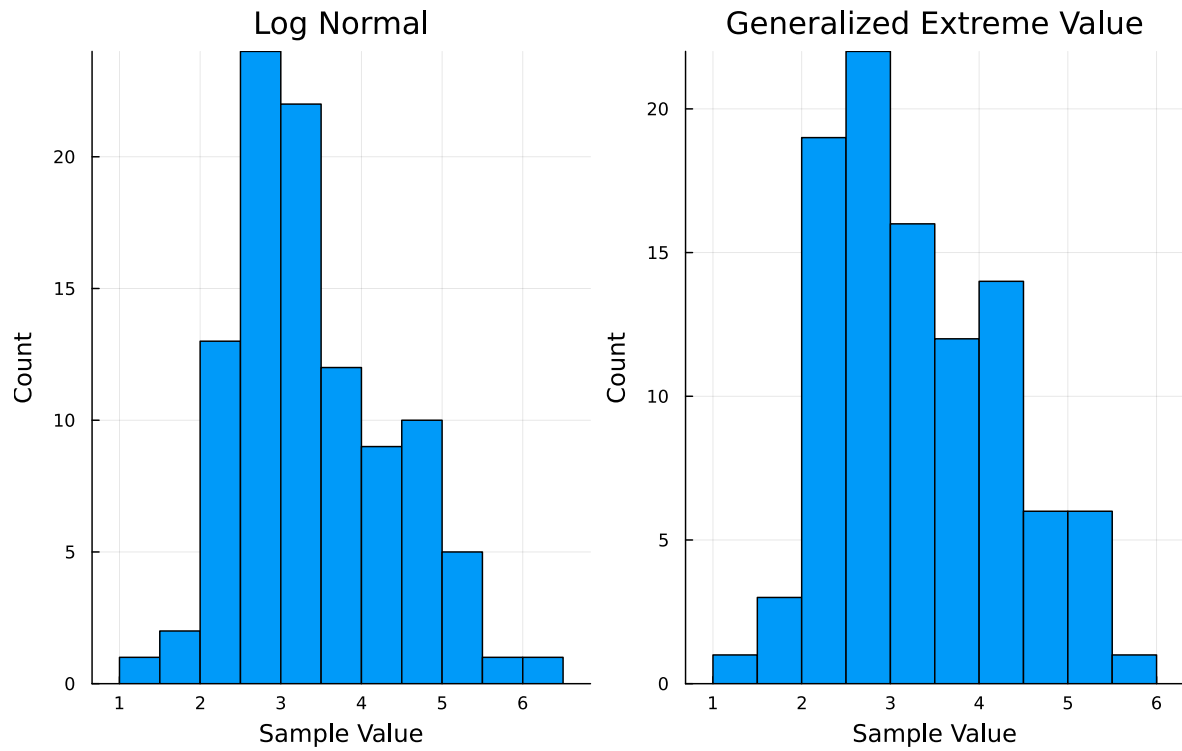
Plot histograms of samples from these distributions and conduct a Monte Carlo experiment for annual maximum flood damages using each. Answer the following questions:

- What are the Monte Carlo estimates of the expected value and the 99% quantile for the annual maximum damage the structure would suffer for each of these flood hazard distributions?
- How did you decide on your sample size?
- Why do you think the estimates differed or did not differ (you can also plot the depth-damage function to compare with the samples)?
- How might you decide (based on getting additional information, if needed) which distribution is “better”?

```
log_dist = LogNormal(1.2, 0.3)
log_sample = rand(log_dist, 100)
p1 = histogram(log_sample, legend=:false, bins=10, title = "Log Normal")

gen_dist = GeneralizedExtremeValue(3, 0.9, -0.15)
gen_sample = rand(gen_dist, 100)
p2 = histogram(gen_sample, legend=:false, bins=10, title = "Generalized Extreme Value")

plot(p1, p2, layout=(1,2), size = (800,500), )
ylabel!("Count")
xlabel!("Sample Value")
```



-> To plot the two distributions above, I used the same method as shown previously to plot the results of a random dice being rolled 100 times. To display the two plots side-by-side I added a “layout = (1,2)” term to my plot command.

## References

Put any consulted sources here, including classmates you worked with/who helped you.